Processes descriptors and two way communication

Tacu Igor, FAF 242
Laboratory Works 7 and 8, Operating Systems
New reproduction on 9 November 2026

Question and execution environment

A shell pipeline and a child that replaces itself with another program both depend on state that survives process creation or program replacement. Lab 7 isolates that state under Linux: PIDs, wait status, open descriptors, and pipe endpoints. Lab 8 applies the same ideas to xv6, where the user API and kernel implementation are small enough to inspect together. The revised experiments below were run in Ubuntu 24.04 x86_64 over SSH and in QEMU with the committed xv6 source. The output is from the new run, not the earlier SSH screenshots.

All six requested programs are present in this repository. Linux sources are in `Lab7-8/linux`; xv6 sources are `xv6/user/run.c` and `xv6/user/pingpong.c`. The complete xv6 tree includes the UPROGS registrations. Raw transcripts are in `Lab7-8/evidence`, and the two runner scripts under `scripts` retain the commands that produced them.

Lab 7 Replacing a child while preserving its identity

`run.c` flushes the parent's initial message before calling `fork`. Otherwise buffered output could be copied and later printed twice when stdout is redirected. The child prints its identity, flushes that output, and invokes `execvp` for an explicit command or `execlp` for the default `ls -l`. The parent waits for the particular child PID with `waitpid`, then distinguishes normal exit from signal termination before interpreting the status bits.

The default listing, a UTC date command, and a deliberately absent program exercise three paths. Both successful programs return status zero. The absent executable returns from exec, prints the error, and calls `_exit(127)`. The parent subsequently reports 127 for the child; the parent utility itself returns normally after reporting that result. It would be misleading to describe the parent's own exit code as 127 simply because that number appears in its message.

```
+ ./run
parent: my PID is 3310792
child: my PID is 3310793, my parent is 3310792
total 80
-rwxr-xr-x 1 root root 16528 Nov  9 14:21 pipeline
-rwxr-xr-x 1 root root 16400 Nov  9 14:21 redirect
-rwxr-xr-x 1 root root 16440 Nov  9 14:21 run
-rwxr-xr-x 1 root root 16440 Nov  9 14:21 zombie
parent: child 3310793 ended with status 0
+ ./run date -u +%Y-%m-%d
parent: my PID is 3310794
child: my PID is 3310795, my parent is 3310794
2026-11-09
parent: child 3310795 ended with status 0
+ ./run no_such_igor_program
parent: my PID is 3310796
child: my PID is 3310797, my parent is 3310796
exec: No such file or directory
parent: child 3310797 ended with status 127
```

Successful exec does not create an additional child and does not run the fallback error branch. It replaces the calling child's address space while retaining its PID and inherited descriptors. This is

why the listing appears between the child's identification and the parent's completion message. The exact PID values are incidental; the relationship between them is the part under test.

Lab 7 Delaying collection of a terminated child

`zombie.c` uses the same exec path but makes the parent sleep before `waitpid`. The child can finish its listing while the parent is still alive. The process query selects children of the saved background parent PID, which avoids mixing this case with unrelated processes on the machine. The first query found a `Z` entry for ls; after waiting for the parent utility, the second query contained only the header.

```
+ parent_pid=3310798
+ ./zombie
+ sleep 1
+ ps -o pid,ppid,stat,cmd --ppid 3310798
    PID    PPID STAT CMD
3310800 3310798 Z    [ls] <defunct>
+ wait 3310798
+ ps -o pid,ppid,stat,cmd --ppid 3310798
    PID    PPID STAT CMD
```

The zombie is not a program still executing the ls instructions. It is the kernel's retained termination information awaiting collection. The delay makes this normally short interval observable. Using `$!` immediately after launching the parent and filtering with `--ppid` ties the observed entry to the experiment rather than to a coincidentally named process.

Lab 7 Redirecting output by descriptor reuse

The redirection child closes descriptor 1 before opening `out.txt`. Since open chooses the lowest available descriptor, the new descriptor is 1. The later ls image writes through the standard-output number it always uses, but that number now refers to the file. The diagnostic is deliberately written to stderr and remains visible outside the file. The parent waits and checks the child's status before announcing completion.

The run reported `open gave descriptor 1`; reading the first lines of `out.txt` displayed the saved listing. To redirect errors instead, close descriptor 2 before opening the error file. The listing showed `out.txt` as empty at the moment ls inspected its metadata; ls was itself generating the bytes subsequently read by head.

Lab 7 A pipe whose reader needs every writer to close

`pipeline.c` creates one pipe and two children. The ls child duplicates the write endpoint onto stdout. The wc child duplicates the read endpoint onto stdin. Each child closes both original endpoints after duplication, and the parent also closes its copies before collecting both children. The ordinary program and `ls /etc | wc -l` each returned **215** in this environment. This is a measurement of the current server's `/etc`, not a fixed expected value for Ubuntu.

The revised program includes a bounded `--delay-eof` mode. It retains the parent's write endpoint for three seconds, then closes it. This demonstrates the reader's dependence on the last writer without leaving a permanently blocked process behind. The measured elapsed time was about three seconds, and wc eventually returned the same count.

```
+ ./pipeline
215
+ ls /etc
+ wc -l
215
+ ./pipeline --delay-eof
parent keeps the write end open for 3 seconds; wc waits for EOF
215

real    0m3.003s
user    0m0.001s
sys 0m0.006s
+ python3 /tmp/igor-os-review-20261109/scripts/check-broken-pipeline.py /tmp/igor-os-review
    -20261109
Part 4.3: pipeline is still running after two seconds
    PID    PPID STAT CMD
3310882 3310880 S    wc -l
Interrupted with SIGINT (Ctrl-C equivalent); return code -2
```

The handout's Part 4.3 was also executed literally: the parent's final `close(p[1])` was omitted in a test variant. After two seconds the parent was still running and ps showed wc waiting. The test then interrupted the process group with SIGINT, the terminal Ctrl-C signal. Its record is included in the block above. The bounded delay mode is an additional experiment, not a replacement for this broken case.

A pipe read can reach EOF only after buffered bytes have been consumed and every descriptor referring to its write end has been closed. Copies inherited through fork count too, which is why closing endpoints only in the producer is insufficient.

Lab 8 Reusing the lifecycle inside xv6

The xv6 `run` utility defaults to echo but also accepts a command and its arguments. This makes success and failure reproducible without editing source between each test. It calls fork, uses exec in the child, and waits in the parent. `run` printed `hello from the child` and status zero; `run ls` listed the generated filesystem; `run no_such_igor_program` took the fallback branch and reported status one.

```
$ run
parent: my pid is 87
child: my pid is 88
hello from the child
parent: child 88 ended with status 0
$ run no_such_igor_program
parent: my pid is 91
child: my pid is 92
exec failed
parent: child 92 ended with status 1
```

The Linux fallback uses 127 and the xv6 fallback uses 1 because those values are explicitly chosen by the utilities. Exec failure does not force every program or OS to choose the same status. The xv6 wait API also differs from Linux waitpid: it collects a child and writes the status through a pointer without selecting a particular PID in this program.

Lab 8 Passing a byte in each direction

`pingpong.c` creates separate parent-to-child and child-to-parent pipes before forking. The parent sends one byte on the first pipe and blocks waiting on the second. The child reads the byte, prints its ping message, and returns the byte on the other pipe. Both processes close the endpoints they do

not use; the parent then waits to collect the child. The implementation checks every one-byte read and write instead of assuming a transfer succeeded.

```
$ pingpong
94: received ping
93: received pong
$
```

The observed ping-before-pong order follows from the data dependency. Specifically, `read(c2p[0], &b, 1)` blocks the parent until a reply byte is available; the later wait collects the terminated child. Two pipes give each direction its own stream and make ownership explicit.

Interpretation and source provenance

These tests connect shell behavior to specific lifetime rules. Fork copies descriptor references, exec preserves them, wait collects termination state, and close can determine whether another process is allowed to finish. The delayed EOF experiment is particularly useful because its final count is unchanged while its completion time changes. It distinguishes a communication-lifetime error from an incorrect data-processing result.

The runner checks compilation with `-Wall -Wextra -Werror`, verifies the Linux cases, builds the complete xv6 image, and checks distinctive output for run and pingpong. It does not claim that this limited set proves arbitrary error recovery or all scheduling orders. MIT xv6 remains the upstream teaching system; the added utilities and their test fixtures are identified in `xv6/PROVENANCE.md` rather than presenting the entire kernel as newly authored code.

Execution captures and code excerpts

The following screenshots show the read-only console of saved SSH execution records and source excerpts. They are linked to the raw recordings in the repository and retain an output hash for traceability. The complete source files accompany the report.

lab7-run

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Lab7-8/linux/run.c

Source excerpt

```
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        if (argc > 1)
            execvp(program, &argv[1]);
        else
            execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        _exit(127);
    }

    int status;
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status))
        printf("parent: child %d ended with status %d\n", pid,
WEXITSTATUS(status));
```

Recorded command output

```
+ ./run
parent: my PID is 3310792
child: my PID is 3310793, my parent is 3310792
total 80
-rwxr-xr-x 1 root root 16528 Nov  9 14:21 pipeline
-rwxr-xr-x 1 root root 16400 Nov  9 14:21 redirect
-rwxr-xr-x 1 root root 16440 Nov  9 14:21 run
-rwxr-xr-x 1 root root 16440 Nov  9 14:21 zombie
parent: child 3310793 ended with status 0
+ ./run date -u +%Y-%m-%d
parent: my PID is 3310794
child: my PID is 3310795, my parent is 3310794
2026-11-09
parent: child 3310795 ended with status 0
+ ./run no_such_igor_program
parent: my PID is 3310796
child: my PID is 3310797, my parent is 3310796
exec: No such file or directory
parent: child 3310797 ended with status 127
```

Output SHA256 df514b5dc31a128774653dd0

Figure 1: Linux run code and execution

lab7-zombie

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Lab7-8/linux/zombie.c

Source excerpt

```
sleep(20);
    int status;
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status))
        printf("parent: child %d ended with status %d\n", pid,
WEXITSTATUS(status));
```

Recorded command output

```
+ parent_pid=3310798
+ ./zombie
+ sleep 1
+ ps -o pid,ppid,stat,cmd --ppid 3310798
    PID    PPID STAT CMD
3310800 3310798 Z    [ls] <defunct>
+ wait 3310798
+ ps -o pid,ppid,stat,cmd --ppid 3310798
    PID    PPID STAT CMD
```

Output SHA256 70257d7a3226da00749d88c8

Figure 2: Zombie collection code and process table

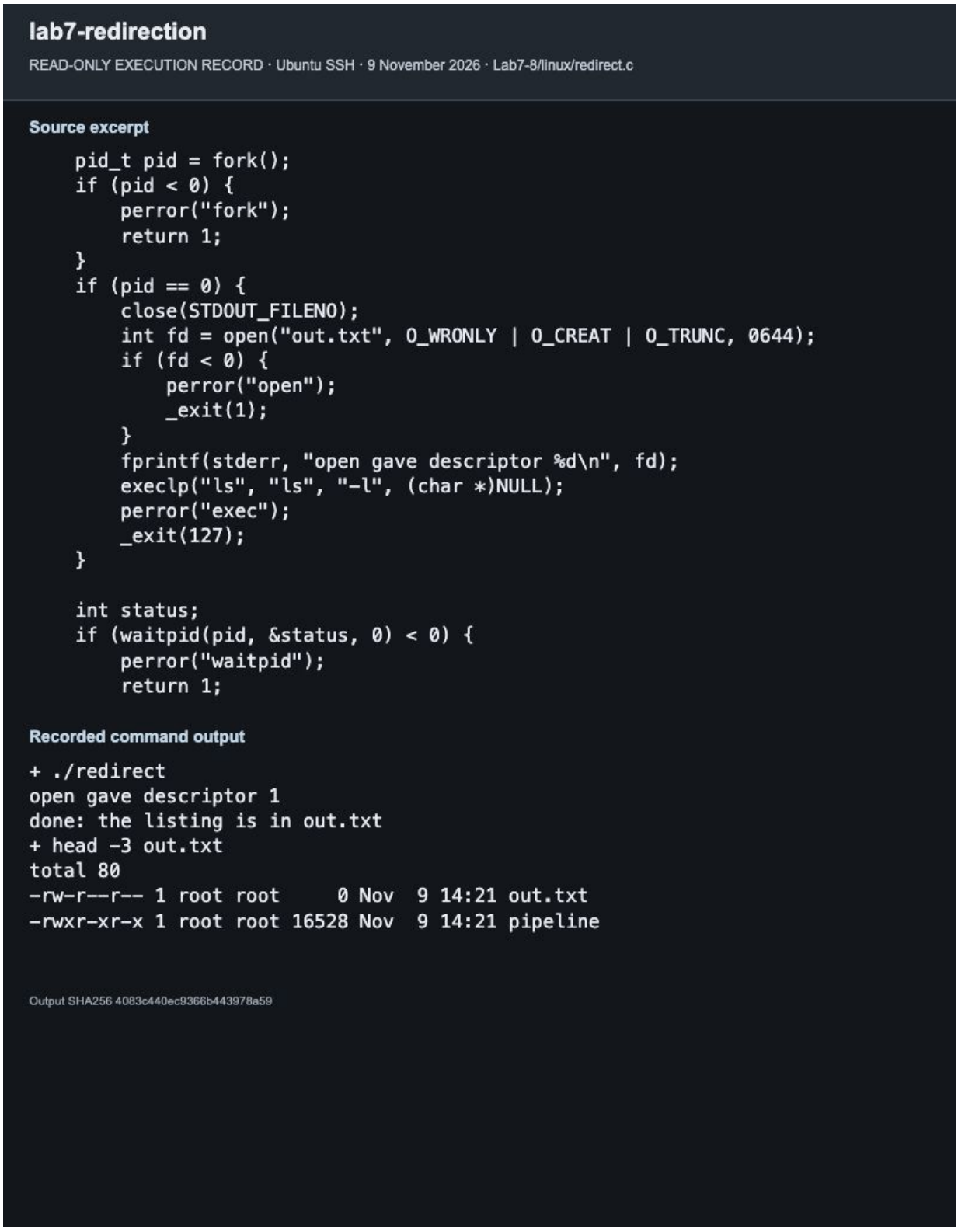


Figure 3: Descriptor redirection code and file output

lab7-pipeline

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Lab7-8/linux/pipeline.c

Source excerpt

```
        return 1;
    }
    if (ls_pid == 0) {
        close(STDOUT_FILENO);
        if (dup(p[1]) != STDOUT_FILENO) _exit(1);
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        _exit(127);
    }

    pid_t wc_pid = fork();
    if (wc_pid < 0) {
        perror("fork");
        close(p[0]);
        close(p[1]);
        waitpid(ls_pid, NULL, 0);
        return 1;
    }
    if (wc_pid == 0) {
        close(STDIN_FILENO);
        if (dup(p[0]) != STDIN_FILENO) _exit(1);
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
```

Recorded command output

```
+ ./pipeline
215
+ ls /etc
+ wc -l
215
+ ./pipeline --delay-eof
parent keeps the write end open for 3 seconds; wc waits for EOF
215

real    0m3.003s
user    0m0.001s
sys     0m0.006s
+ python3 /tmp/igor-os-review-20261109/scripts/check-broken-pipeline.py
/tmp/igor-os-review-20261109
Part 4.3: pipeline is still running after two seconds
    PID    PPID STAT CMD
3310882 3310880 S    wc -l
Interrupted with SIGINT (Ctrl-C equivalent); return code -2
```

Output SHA256 2c891d5adf5d0135488c811b

Figure 4: Pipe endpoints and the omitted close experiment

lab8-run

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · xv6/user/run.c

Source excerpt

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    if(argc > 1){
      exec(argv[1], &argv[1]);
    } else {
      char *args[] = { "echo", "hello", "from", "the", "child", 0 };
      exec("echo", args);
    }
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Recorded command output

```
$ run
parent: my pid is 87
child: my pid is 88
hello from the child
parent: child 88 ended with status 0
$ run no_such_igor_program
parent: my pid is 91
child: my pid is 92
exec failed
parent: child 92 ended with status 1
```

Output SHA256 dbd903f460bf63e47866cd0d

Figure 5: xv6 run code and execution

lab8-pingpong

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · xv6/user/pingpong.c

Source excerpt

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  int p2c[2], c2p[2];
  char b = 'x';
  if(pipe(p2c) < 0 || pipe(c2p) < 0){
    printf("pipe failed\n");
    exit(1);
  }
  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){
    close(p2c[1]);
    close(c2p[0]);
    if(read(p2c[0], &b, 1) != 1) exit(1);
    printf("%d: received ping\n", getpid());
    if(write(c2p[1], &b, 1) != 1) exit(1);
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  }
  close(p2c[0]);
  close(c2p[1]);
  if(write(p2c[1], &b, 1) != 1) exit(1);
  if(read(c2p[0], &b, 1) != 1) exit(1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

Recorded command output

```
$ pingpong
94: received ping
93: received pong
$
```

Output SHA256 34f3ed2c182621ae9514adbd

Figure 6: Two pipes in the code and the ordered reply